Lecture 9: The Curse of Dimensionality and Principal Component Analysis

Through a Two-Sensor Problem

AI 501: Mathematics for Artificial Intelligence

Fall 2026

Abstract

Real data often has many features, but high-dimensional spaces behave strangely: they are mostly empty, and almost all of their volume sits near the surface. The saving grace is that real data usually occupies only a thin, low-dimensional part of the space. Principal component analysis (PCA) finds that part. The idea in one sentence: *rotate the axes to line up with the directions in which the data varies most, and keep only the first few.* One small example, ten readings from two nearly identical sensors, is carried through every step, and the notes end with the link to the SVD of Lecture 8 and the main limitations of PCA.

Contents

1 One problem: two sensors that say almost the same thing **1**

2 The curse of dimensionality **2**

3 Dimensionality reduction by feature extraction **4**

4 Mean and covariance **4**

5 Why the eigenvectors of the covariance matrix **5**

6 The algorithm and the choice of k **6**

7 The full example: the two sensors **7**

8 Geometric interpretation **8**

9 Practical considerations and limitations **9**

10 Applications **11**

11 Summary map **11**

---

1 One problem: two sensors that say almost the same thing

Two sensors measure the same quantity, and each reading is a pair $x = (x_1, x_2)^T$. Ten readings are collected:

i	x_1	x_2	i	x_1	x_2
1	2.5	2.4	6	2.3	2.7
2	0.5	0.7	7	2.0	1.6
3	2.2	2.9	8	1.0	1.1
4	1.9	2.2	9	1.5	1.6
5	3.1	3.0	10	1.1	0.9

Figure 1 plots them. The points hug a line: when one sensor reads high, so does the other. Two numbers per reading seem like more than we need.

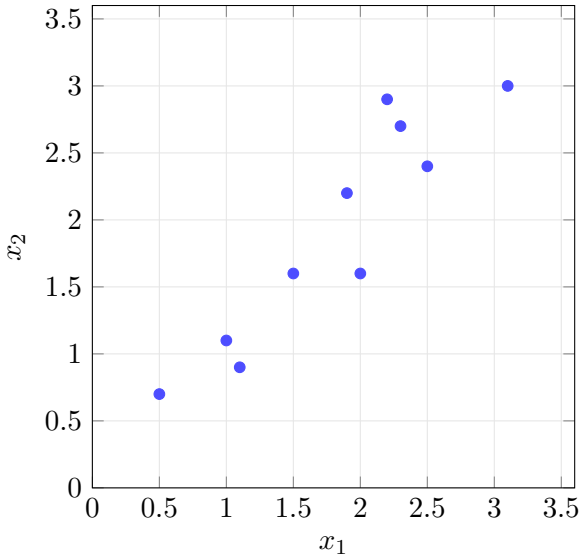


Figure 1: The ten readings. They lie close to a line, so the pair (x_1, x_2) is nearly redundant.

Our goal is to describe each reading by *one* number and lose as little as possible. The natural choice is the position along the line that the points hug: it carries almost all of the information, while the distance from the line is mostly noise. PCA is a systematic way to find such a line (and, in higher dimensions, such a plane or subspace). We ask: why are many features a problem, and why can we still succeed (Section 2)? How do we replace d numbers by $k < d$ and lose as little as possible (Sections 3–7)? What is the picture behind it, and when does it fail (Sections 8–9)?

2 The curse of dimensionality

The core fact is that *the space grows exponentially with the number of dimensions* d, so the number of data points needed to cover it must grow exponentially too.

Counting cells. Cut each feature axis into m bins. In d dimensions this gives m^d cells (Figure 2). To have a training point in every cell we need $n \geq m^d$ points. With $m = 10$:

$$d = 2: \ 10^2 = 100, \qquad d = 3: \ 10^3 = 1000, \qquad d = 10: \ 10^{10}, \qquad d = 784: \ 10^{784}.$$

The last case is a small 28×28 image. No dataset can fill that many cells, so a new point almost never has a training point nearby.

Volume near the surface. Doubling the radius of a disc multiplies its area by 2^2, and doubling the radius of a ball multiplies its volume by 2^3. In d dimensions, multiplying the radius

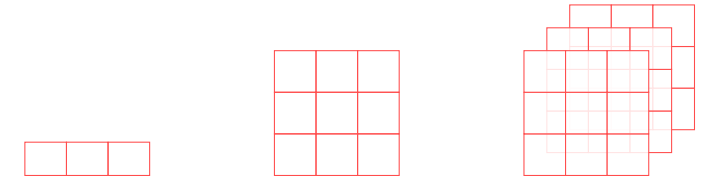


Figure 2: With 3 bins per axis the number of cells is 3^d. To have data in every cell we need exponentially many samples.

by r multiplies the volume by r^d, so a ball of radius r has volume $K_d r^d$, where K_d depends only on d. Now ask how much of the unit ball lies in a thin shell of width ϵ just inside the surface, between the radii $1-\epsilon$ and 1. The inner ball has volume $K_d(1-\epsilon)^d$, so the fraction in the shell is

$$\frac{K_d - K_d(1-\epsilon)^d}{K_d} = \boxed{1-(1-\epsilon)^d}.$$

In words: the factor $(1-\epsilon)$ is smaller than 1, and raising it to a large power d makes it tiny, so almost nothing is left in the inner ball. For $\epsilon = 0.1$:

d	1	2	10	50	400	784
in the shell, $1-0.9^d$	0.1	0.19	0.65	0.995	1.000	1.000
in the inner ball, 0.9^d	0.9	0.81	0.35	0.005	0.000	0.000

For $d = 400$ essentially *all* the volume is within 10% of the surface, so nearly every point is at almost the same distance from the centre. When all points look equally far, "nearby" loses its meaning, and methods that rely on distances to neighbours suffer.

Check yourself. What fraction of a 100-dimensional ball's volume lies within $\epsilon = 0.05$ of the surface?
Answer. $1 - 0.95^{100} \approx 1 - 0.006 = 0.994$: more than 99%.

Why it is not hopeless. High-dimensional space is empty, but *real data does not fill it.* It lives in a region of much lower effective dimension: points near a line or a plane (Figure 3), or the pictures of a rotating object, which trace out a curve inside a space of a million pixels. *Dimensionality reduction* finds that region and works there.

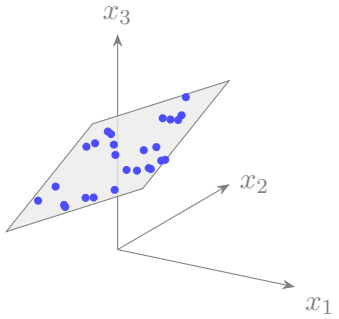


Figure 3: Points in $\mathbf{R}^3$ that lie close to a plane. Although $d = 3$, two coordinates along the plane describe them almost completely.

3 Dimensionality reduction by feature extraction

Feature extraction replaces the d original features by a smaller set of $k < d$ new ones, built from the old ones. Different goals give different methods: *minimise information loss* gives **principal component analysis (PCA)**; *maximise class separation* gives linear discriminant analysis (LDA), which we mention only for contrast. PCA uses a *linear* map because it is simple and tractable:

$$z = W^T x, \qquad x \in \mathbf{R}^d, \quad W \in \mathbf{R}^{d\times k}, \quad W^T \in \mathbf{R}^{k\times d}, \quad z \in \mathbf{R}^k.$$

In words: each column $v_j \in \mathbf{R}^d$ of W is a direction (a unit vector, perpendicular to the others), and the jth new feature $z_j = v_j^T x$ is the coordinate of x along that direction, that is, where x lands when we drop a perpendicular onto the line through v_j. The directions are the *principal components*. So PCA is a change of axes, followed by keeping only k of the new axes.

Which axes should we keep? We want the new features to be (1) of *large variance*, so that they capture most of the variation in the data and distinguish one sample from another, and (2) *uncorrelated*, so that none of them repeats what another already says. Sections 4 and 5 show that the eigenvectors of the covariance matrix achieve both.

Check yourself. What is the size of W for a planar ($k = 2$) summary of 100-dimensional data?
Answer. W is 100×2, W^T is 2×100, and $z \in \mathbf{R}^2$.

4 Mean and covariance

To find the direction in which the data varies most, we first need a way to measure how the features vary, alone and together. We have n samples $x_1, \ldots, x_n \in \mathbf{R}^d$ (here $d = 2$, $n = 10$).

1. **Mean.** $\bar{x} = \frac{1}{n}\sum_{i=1}^n x_i \in \mathbf{R}^d$: the centre of the cloud. For our data $\bar{x} = (1.81,\ 1.91)^T$.
2. **Center.** $s_i = x_i - \bar{x}$: shift the cloud so that its centre sits at the origin. Variation is measured around the centre, so this is the natural starting point. Stack the s_i as the columns of $S = [s_1, \ldots, s_n] \in \mathbf{R}^{d\times n}$ (one *column* per sample).
3. **Covariance.** $C = \dfrac{1}{n}\displaystyle\sum_{i=1}^{n} s_i s_i^T = \frac{1}{n} SS^T \in \mathbf{R}^{d\times d}$: a table of how the features vary and how they vary together.

(We write C rather than Σ to avoid confusion with the singular values of Lecture 8. Some authors divide by $n - 1$; that multiplies all eigenvalues by the same factor and changes nothing else.)

What the entries mean. Entry (j, k) of C averages the product of the deviations of feature j and feature k from their means. If the two deviations tend to have the same sign (both above average, or both below), the product is positive and so is the covariance; if one tends to be high when the other is low, it is negative; if they have nothing to do with each other, the positive and negative products cancel and the covariance is near zero. The diagonal entries C_{jj} are the covariance of a feature with itself, which is its *variance*: how much that feature spreads. Their sum $\operatorname{tr} C$ is the *total variance*. For the sensors

$$C = \begin{bmatrix} 0.5549 & 0.5539 \\ 0.5539 & 0.6449 \end{bmatrix}, \qquad \operatorname{tr} C = 1.1998, \qquad \text{correlation } \rho = \frac{0.5539}{\sqrt{0.5549 \cdot 0.6449}} \approx 0.926.$$

Each sensor spreads by about 0.6 (the diagonal), and the covariance 0.55 is nearly as large as the variances themselves: when one sensor is above its average, so is the other. The correlation ρ rescales the covariance to a number between -1 and 1; 0.926 is close to 1, the signature of the line in Figure 1.

Key fact. C is symmetric and positive semi-definite, so it has real eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$ and an orthonormal basis of eigenvectors.

Why. $C = \frac{1}{n}SS^T$ is a Gram matrix (Lecture 8): $x^TCx = \frac{1}{n}\sum_i (s_i^T x)^2 \geq 0$. **In words:** the quadratic form of C turns out to be a *variance* (Section 5), and a variance can never be negative.

Check yourself. The second sensor always reads exactly twice the first, and $\mathrm{var}(x_1) = 1$. What are C and its eigenvalues?
Answer. $C = [\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}]$, with trace 5 and determinant 0, so $\lambda = 5, 0$: all variation is along $(1,2)^T/\sqrt{5}$ and the second feature adds nothing.

5 Why the eigenvectors of the covariance matrix

PCA takes the eigen-decomposition $C = VDV^T$, with $V = [v_1, \ldots, v_d]$ orthogonal and $D = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$ sorted $\lambda_1 \geq \cdots \geq \lambda_d$, and keeps the eigenvectors with the largest eigenvalues. This section explains why that is the right recipe, in three steps: the eigenvalues are the spread of the data along the eigenvectors; the new features are uncorrelated; and the information lost is easy to compute.

5.1 Spread along a direction

Choose a direction, a unit vector w, and drop every centered sample onto the line through w. Each sample s_i lands at the coordinate $a_i = w^T s_i$: its "shadow" on that line. The shadows have mean 0 and variance

$$\frac{1}{n}\sum_{i=1}^{n} a_i^2 = \frac{1}{n}\sum_{i=1}^{n} w^T s_i s_i^T w = w^T C w.$$

In words: w^TCw measures how spread out the shadows are, that is, how much the data varies along the direction w. A direction with large w^TCw separates the samples well; a direction with small w^TCw squashes them together. Which direction is best?

Key fact 1 (direction of largest variance). Among all unit vectors w, the variance w^TCw is largest for $w = v_1$ and equals λ_1. The next best direction perpendicular to v_1 is v_2 with variance λ_2, and so on. In general, λ_j *is the variance along* v_j.

Why. Describe w in the eigenvector basis, $w = \sum_i c_i v_i$ with $\sum c_i^2 = 1$. As in Lecture 8, $w^TCw = \sum_i \lambda_i c_i^2$. **In words:** this is a weighted average of the eigenvalues (the weights c_i^2 are non-negative and add up to 1). An average cannot exceed the largest value being averaged, so $w^TCw \leq \lambda_1$, and equality holds when all the weight sits on λ_1, that is, when $w = v_1$. For the next direction, we require $w \perp v_1$, which forces $c_1 = 0$ and leaves λ_2 as the largest available value, and so on.

So the principal components, found one after another, each the best direction perpendicular to the ones before, are the eigenvectors in decreasing order of eigenvalue.

5.2 The new features are uncorrelated

Take $W = [v_1, \ldots, v_k]$ and the scores $z = W^T s$. Their covariance matrix is W^TCW, whose (i,j) entry is $v_i^TCv_j = \lambda_j v_i^T v_j$. Since eigenvectors of a symmetric matrix are perpendicular, $v_i^T v_j = 0$ for $i \neq j$, so

$$\mathrm{Cov}(z) = W^TCW = \mathrm{diag}(\lambda_1, \ldots, \lambda_k).$$

In words: the off-diagonal entries are zero, so the new features are *uncorrelated*, and the diagonal entries are the largest variances available. This is what "PCA decorrelates the features"

means, and it is the reason we use *orthogonal* directions: no new feature repeats what another says.

5.3 What is lost

Suppose we keep only k of the new coordinates and rebuild each sample from them: $\hat{x}_i = \bar{x} + W z_i$. How far is $\hat{x}_i$ from the true x_i?

Key fact 2 (information lost). The average squared reconstruction error is

$$\frac{1}{n}\sum_{i=1}^{n} \|x_i - \hat{x}_i\|^2 = \sum_{j=k+1}^{d} \lambda_j = \operatorname{tr} C - \sum_{j=1}^{k} \lambda_j,$$

the sum of the discarded eigenvalues: the variance we did not keep.

Why. Look at one centered sample s_i (Figure 5 below shows the picture for $k = 1$). Rebuilding it from its scores puts it at the point p_i on the kept directions, its shadow. The leftover $r_i = s_i - p_i$ is the part we lost, and it points *perpendicular* to the kept directions, so p_i and r_i form a right angle. By Pythagoras,

$$\|s_i\|^2 = \|p_i\|^2 + \|r_i\|^2.$$

In words: the squared distance of a point from the centre splits into a part we keep and a part we lose. Averaging over all samples, *total variance = variance kept + error*. The total variance is $\operatorname{tr} C = \lambda_1 + \cdots + \lambda_d$, and the variance kept is the sum of the variances along the kept directions, $\lambda_1 + \cdots + \lambda_k$ (Key fact 1). What remains is the error, $\lambda_{k+1} + \cdots + \lambda_d$.

So *minimising the loss and maximising the variance kept are the same problem*: since the total is fixed, the less variance we throw away, the smaller the error. Keeping the directions of largest variance is exactly what makes the reconstruction error smallest.

Check yourself. Can any direction in the sensor data have variance larger than $\lambda_1 = 1.1556$? What is the variance along $w = (1, 0)^T$?
Answer. No: λ_1 is the maximum. Along $(1, 0)^T$ the variance is $C_{11} = 0.5549$, which is smaller.

6 The algorithm and the choice of k

1. **Center.** $\bar{x} = \frac{1}{n}\sum_i x_i$, $s_i = x_i - \bar{x}$, $S \in \mathbf{R}^{d\times n}$. *(Move the cloud's centre to the origin.)*
2. **Covariance.** $C = \frac{1}{n} S S^T \in \mathbf{R}^{d\times d}$. *(Measure how the features vary and co-vary.)*
3. **Eigen-decomposition.** $C = V D V^T$, eigenvalues sorted $\lambda_1 \geq \cdots \geq \lambda_d$. *(Find the directions of spread: the columns v_j are the principal components.)*
4. **Select.** $W = [v_1, \ldots, v_k] \in \mathbf{R}^{d\times k}$. *(Keep the k directions with the most spread.)*
5. **Reduce.** $z = W^T(x - \bar{x}) \in \mathbf{R}^k$. *(Coordinates along the kept directions.)* **Reconstruct (if needed).** $\hat{x} = \bar{x} + W z$.

Two remarks on the slides. (i) The slides write $z = W^T x$; that equals our formula only if the data are already centered. For raw data subtract $\bar{x}$ first (and add it back when reconstructing). (ii) Software often returns eigenvalues in *ascending* order, so the top component is then the *last* column. In the slides' example $D = \operatorname{diag}(0.0442, 1.1556)$.

Choosing k. By Key fact 2, the fraction of the variance that survives is

$$T_k = \frac{\lambda_1 + \cdots + \lambda_k}{\lambda_1 + \cdots + \lambda_d}.$$

In words: the share of the total spread that the first k components carry. We choose the smallest k with T_k above a threshold (for example 0.8 means 80% of the variation is preserved). If the eigenvalues drop quickly, a small k suffices, which is the sign that the data live near a low-dimensional region.

7 The full example: the two sensors

We run the algorithm on the sensor data with $d = 2$, $n = 10$, $k = 1$. Steps 1 and 2 were done in Section 4: $\bar{x} = (1.81, 1.91)^T$ and $C = \left[\begin{smallmatrix} 0.5549 & 0.5539 \\ 0.5539 & 0.6449 \end{smallmatrix}\right]$.

Step 3: the directions of spread. The characteristic polynomial of C is $\lambda^2 - 1.1998\lambda + 0.0511$, so

$$\lambda = \tfrac{1}{2}\big[1.1998 \pm \sqrt{0.0081 + 1.2272}\big] = \tfrac{1}{2}\big[1.1998 \pm 1.1114\big], \qquad \lambda_1 = 1.1556, \quad \lambda_2 = 0.0442.$$

(Check: $\lambda_1 + \lambda_2 = 1.1998 = \operatorname{tr} C$.) Solving $(C - \lambda_1 I)v = 0$ and normalising,

$$v_1 = \begin{bmatrix} 0.6779 \\ 0.7352 \end{bmatrix}, \qquad v_2 = \begin{bmatrix} -0.7352 \\ 0.6779 \end{bmatrix}.$$

Read the answer: the data spread by 1.1556 along v_1 and by only 0.0442 along v_2, about 26 times less. The first component points 47.3° above the x_1 axis, almost the diagonal: it measures the *overall level* of the two sensors. The second is perpendicular to it: the *difference* between the sensors, which is nearly constant.

Steps 4 and 5: keep one direction and reduce. $W = [v_1] \in \mathbf{R}^{2\times 1}$, $z_i = v_1^T(x_i - \bar{x})$ and $\hat{x}_i = \bar{x} + z_i v_1$. For sample 1, $s_1 = (0.69, 0.49)^T$, so $z_1 = 0.6779 \cdot 0.69 + 0.7352 \cdot 0.49 = 0.828$ and $\hat{x}_1 = (2.371,\ 2.519)$, close to $x_1 = (2.5,\ 2.4)$. All ten samples:

i	x_i	z_i	$\hat{x}_i$	$\|x_i - \hat{x}_i\|^2$
1	(2.5, 2.4)	0.8280	(2.371, 2.519)	0.0307
2	(0.5, 0.7)	−1.7776	(0.605, 0.603)	0.0204
3	(2.2, 2.9)	0.9922	(2.483, 2.639)	0.1477
4	(1.9, 2.2)	0.2742	(1.996, 2.112)	0.0170
5	(3.1, 3.0)	1.6758	(2.946, 3.142)	0.0439
6	(2.3, 2.7)	0.9129	(2.429, 2.581)	0.0307
7	(2.0, 1.6)	−0.0991	(1.743, 1.837)	0.1224
8	(1.0, 1.1)	−1.1446	(1.034, 1.069)	0.0022
9	(1.5, 1.6)	−0.4380	(1.513, 1.588)	0.0003
10	(1.1, 0.9)	−1.2238	(0.980, 1.010)	0.0265

The squared errors add up to 0.4418, so their mean is $0.0442 = \lambda_2$: exactly the discarded eigenvalue, as Key fact 2 predicts. The variance kept is $T_1 = 1.1556/1.1998 \approx 0.963$: *one number per reading keeps* 96.3% *of the variation.*

Relation to the slides. The scores printed on the slides $(3.4591,\ 0.8536, \ldots)$ are $v_1^T x_i$ computed from the *uncentered* data. They differ from ours by the constant $v_1^T \bar{x} = 2.6311$ (sample 1: $0.8280 + 2.6311 = 3.4591$). The spread is identical; ours have mean zero.

Check yourself. What are the error and T_k if we keep $k = 2$?
Answer. $W = V$ is square and orthogonal, so nothing is lost: $\hat{x}_i = x_i$, error 0, $T_2 = 1$.

8 Geometric interpretation

Change of coordinates, then drop one. Figure 4 shows what PCA does. Left: the centered data with the principal directions v_1, v_2. Middle: the *same* points in the new coordinates $z = V^T s$ (a rotation, which changes no distances). Right: we ignore z_2, so every point moves perpendicularly onto the line through $\bar{x}$ in the direction v_1. Because the cloud was already thin in the z_2 direction, this barely changes anything.

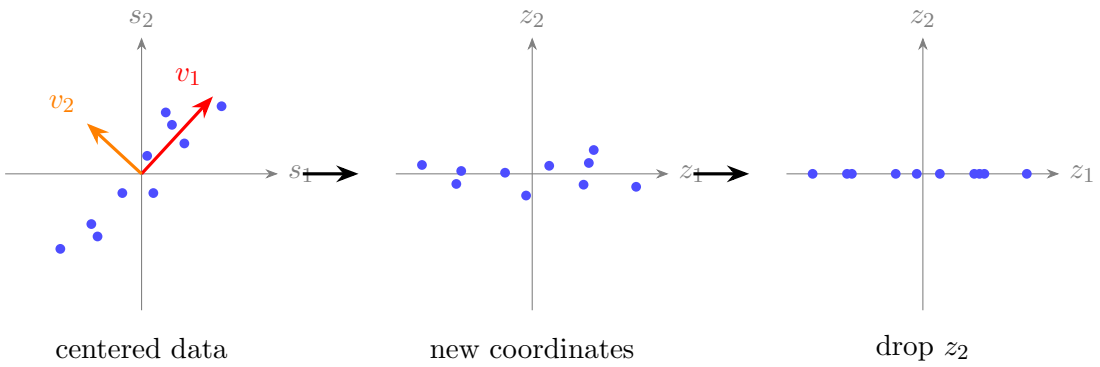


Figure 4: PCA as a change of coordinates followed by dropping the coordinates with little variance. The spread along z_1 (variance 1.156) is much larger than along z_2 (variance 0.044), so dropping z_2 hardly changes the picture.

Projection: what is kept and what is lost. Figure 5 shows the projection onto the line $\bar{x} + tv_1$. The red segments are the errors $r_i = x_i - \hat{x}_i$, and they are *perpendicular* to the line. This is exactly the Pythagoras picture of Key fact 2: for each point, $\|s_i\|^2 = \|p_i\|^2 + \|r_i\|^2$, a part along the line (kept) and a part across it (lost). For sample 1, $0.7162 = 0.6855 + 0.0307$. PCA chooses the line that makes the red segments as short as possible on average, which is the same as making the spread along the line as large as possible.

The principal components are the axes of the data cloud. The cloud is roughly an ellipse whose axes are the principal components and whose semi-axes are $\sqrt{\lambda_j}$: here $\sqrt{1.1556} = 1.075$ along v_1 and $\sqrt{0.0442} = 0.210$ along v_2 (Figure 6), about five times longer than wide. This is the circle-to-ellipse picture of Lecture 8, now drawn around data. The first principal component is the long axis of the cloud, the second is the next one perpendicular to it, and so on. Dropping a component flattens the cloud along one of its axes, and the loss λ_j is the variance that axis carried.

8.1 Link to the SVD

PCA and the SVD ask the same question about the data matrix $S \in \mathbf{R}^{d \times n}$: in which directions is it largest? The SVD of S answers it with the directions along which S stretches the most; PCA answers it with the directions along which the data spread the most. They are the same directions. To see this, let $S = U\Sigma V^T$ be the SVD of S (these are the SVD's symbols, not the eigenvectors V of C). Then

$$C = \frac{1}{n} SS^T = \frac{1}{n} U\Sigma V^T V \Sigma^T U^T = U \left(\frac{\Sigma\Sigma^T}{n} \right) U^T,$$

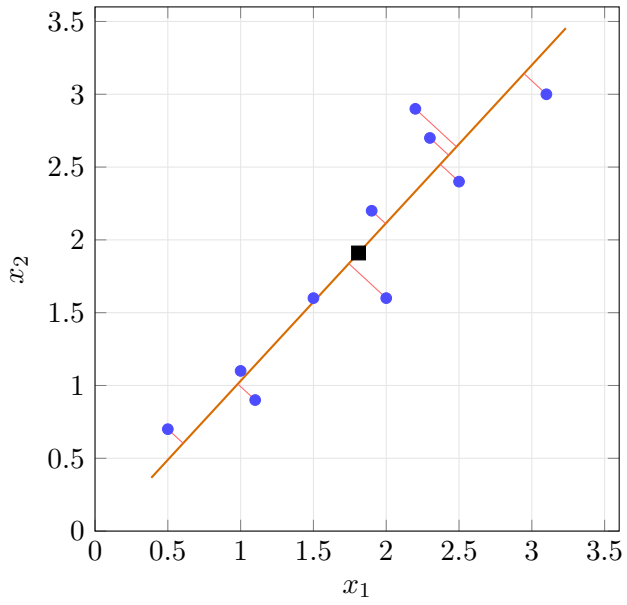


Figure 5: Projection onto the first principal component. Orange: the line through the mean $\bar{x}$ (black square) in direction v_1. Red: the residuals, perpendicular to the line. Blue: the data.

which is an eigen-decomposition of C. Therefore **the principal components are the left singular vectors of** S, and **the eigenvalues are** $\lambda_j = \sigma_j^2/n$. **In words:** the singular value σ_j says how strongly the data stretch along u_j, and dividing the square by n turns it into a variance. Moreover, the reconstruction of the whole data set is the k-rank approximation of S from Lecture 8: keeping k principal components is the same as keeping k layers of the SVD, which is one more way to see why PCA is optimal. For the sensors, S (2×10) has $\sigma = 3.3994,\ 0.6646$, and $\sigma_j^2/10 = 1.1556,\ 0.0442$, the eigenvalues found above.

A note on conventions. Here samples are *columns* of S, so the principal components are the left singular vectors. Many textbooks and software packages store samples as *rows*, $X_c = S^T \in \mathbf{R}^{n \times d}$, in which case $C = \frac{1}{n} X_c^T X_c$ and the principal components are the *right* singular vectors of X_c. It is the same result: the roles of U and V swap when the matrix is transposed. In practice the SVD is preferred over forming C, because computing SS^T squares the numbers and loses small values to rounding.

Check yourself. Which singular values of S correspond to the semi-axes 1.075 and 0.210? *Answer.* The semi-axes are $\sqrt{\lambda_j} = \sigma_j/\sqrt{n}$, so they come from $\sigma_1 = 3.40$ and $\sigma_2 = 0.66$ divided by $\sqrt{10} = 3.16$.

9 Practical considerations and limitations

Scaling matters. PCA looks for variance, and variance depends on units. Suppose sensor 1 reports in millimetres instead of metres, so every x_1 is multiplied by 100. Then $C = \left[\begin{smallmatrix} 5549 & 55.39 \\ 55.39 & 0.6449 \end{smallmatrix}\right]$, the first component is almost exactly the x_1 axis, $v_1 \approx (1.000, 0.010)^T$, and it carries 99.998% of the variance, only because the numbers in that column are bigger. The unit change has hidden the relationship between the sensors.

The usual remedy is to *standardise* each feature (subtract the mean, divide by the standard deviation), so that every feature has variance 1 and no feature wins just because of its units. The covariance matrix of the standardised data is the *correlation* matrix $R = \left[\begin{smallmatrix} 1 & 0.926 \\ 0.926 & 1 \end{smallmatrix}\right]$, which does not depend on the units. Its eigenvalues are $1 \pm \rho = 1.926,\ 0.074$ with eigenvectors $(1, \pm 1)^T/\sqrt{2}$,

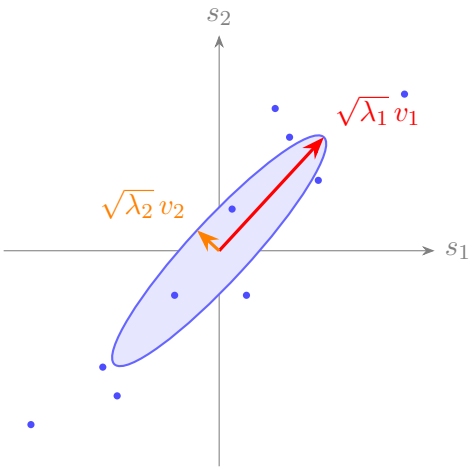


Figure 6: The covariance ellipse $s^T C^{-1} s = 1$ of the sensor data. Its axes are the principal components, its semi-axes $\sqrt{\lambda_j}$.

so the first component is the average of the two standardised sensors and keeps 96.3% of the variance. Standardising is the right default when features have different units.

A limitation: variance is not class separation. PCA never sees class labels. It keeps the directions of large spread, but the direction that separates two classes need not be a direction of large spread. Consider eight points: class A at $x_2 = +1$ and class B at $x_2 = -1$, each with $x_1 \in \{-4, -2, 2, 4\}$. The variances are 10 for x_1 and 1 for x_2, and the covariance is 0, so $C = \mathrm{diag}(10, 1)$ and the first component is the x_1 axis. Projecting onto it puts the two classes on top of each other (Figure 7), while the discarded second component, with only 1/11 of the variance, separates them perfectly. *The direction of maximum variance need not be the direction that matters for classification.*

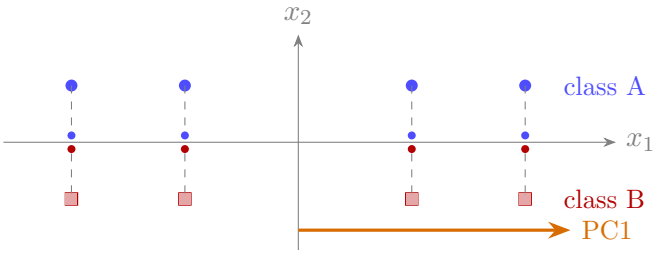


Figure 7: Two classes separated along x_2, with large spread along x_1. The first principal component is the x_1 axis. Projecting onto it (the dots on the axis) puts the classes on top of one another.

The remedy is a method that *uses the labels*, such as LDA. PCA answers "where is the variation?"; LDA answers "where is the difference between the classes?".

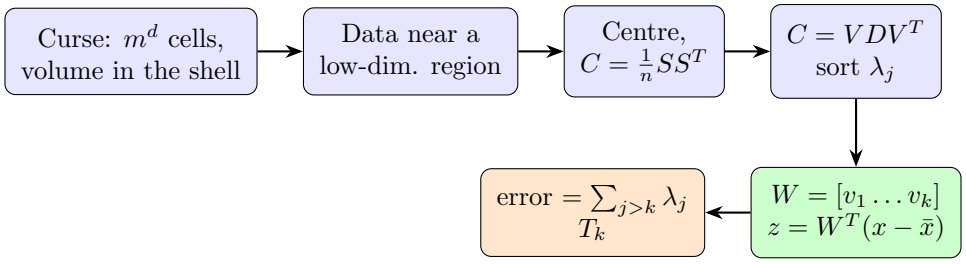
Check yourself. Does standardising the two features of this example fix the problem?
Answer. No. Both features then have variance 1 and covariance 0, so every direction is equally good and PCA has no preferred component. Scaling cannot tell PCA which direction separates the classes.

10 Applications

- **Compression.** Store the scores $z_i \in \mathbf{R}^k$ plus $\bar{x}$ and W, and rebuild $\hat{x}_i = \bar{x} + Wz_i$ when needed. For $n = 10{,}000$ images of $28 \times 28 = 784$ pixels and $k = 50$, the raw data takes 7,840,000 numbers and the compressed form $10{,}000 \cdot 50 + 784 \cdot 50 + 784 = 539{,}984$, about 6.9%.
- **Denoising.** Noise is spread over all directions while the signal sits in a few directions of large variance. Keeping only the top k components removes most of the noise.
- **Visualisation.** With $k = 2$ or 3 the scores can be plotted, so clusters and outliers in high-dimensional data become visible.
- **Features for another model.** A model on k uncorrelated, high-variance features has far fewer parameters and less risk of overfitting. This is PCA's answer to the curse of dimensionality: the data lives near a k-dimensional region, so we describe it with k numbers.

Check yourself. 64×64 images, with the first 30 components keeping 90% of the variance. How many numbers per image after PCA, and how large is W?
Answer. $d = 4096$ and $k = 30$: each image becomes 30 scores, and W is 4096×30.

11 Summary map



Idea	Key fact	In the two-sensor example
Curse of dimensionality	m^d cells; shell fraction $1 - (1-\epsilon)^d$	$d = 400$: all but 5×10^{-19} in the shell
Covariance	$C = \frac{1}{n}SS^T$, symmetric PSD	$C = \left[\begin{smallmatrix} 0.5549 & 0.5539 \\ 0.5539 & 0.6449 \end{smallmatrix}\right]$, $\rho = 0.926$
Directions	v_j eigenvectors of C; λ_j = variance along v_j; scores are uncorrelated	$\lambda = 1.1556,\ 0.0442$; v_1 at 47.3°
Information lost	error $= \sum_{j>k} \lambda_j$; kept T_k	0.0442; $T_1 = 0.963$
Geometry	axes of the data ellipse, semi-axes $\sqrt{\lambda_j}$	1.075 and 0.210
SVD link	$S = U\Sigma V^T$: PCs $= u_j$, $\lambda_j = \sigma_j^2/n$	$\sigma = 3.40,\ 0.66$
Limits	depends on scale; ignores labels	$\times 100$ on x_1 hijacks PC1; classes along a low-variance axis